



Project Guidelines and Research Resources

Project Structure

Your project consists of **three main deliverables**, each contributing to the final grade:

1. Documentation Phase (Literature Review & Problem Analysis)

- **Objective:** Research and understand the state-of-the-art solutions to your assigned problem
- **Requirements:**
 - Search and analyze research articles from academic databases (IEEE Xplore, ACM, arXiv, etc.)
 - Present the problem statement clearly
 - Describe existing solutions and their trade-offs
 - Identify the approach you will implement
 - Include architectural considerations for FPGA/HDL implementation
- **Deliverables:**
 - Written report (10-15 pages)
 - Presentation slides
 - Bibliography with at least 10 research papers

2. Implementation Phase (HDL/HCL Development)

- **Objective:** Implement your chosen algorithm in hardware
- **Requirements:**
 - Create a GitHub repository for your project

- Implement using Hardware Description Language (HDL) such as:
 - VHDL
 - Verilog
 - SystemVerilog
- OR Hardware Construction Language (HCL) such as:
 - Chisel
 - PyMTL
 - Bluespec
 - RHDL
 - HLS (C/C++ to HDL)
- Include testbenches for verification
- Document your code thoroughly
- Provide build/synthesis scripts
- **Deliverables:**
 - GitHub repository with complete source code
 - README with build instructions
 - Test results and waveforms

3. Evaluation Phase (FPGA Synthesis & Performance Analysis)

- **Objective:** Synthesize and evaluate your design on FPGA hardware
- **Requirements:**
 - Synthesize your design using FPGA tools (Vivado, Yosys, etc.)
 - Report resource utilization:
 - Number of LUTs (Look-Up Tables)
 - Number of DSP blocks used
 - Block RAM utilization
 - Flip-flops
 - Analyze performance metrics:
 - Maximum clock frequency
 - Latency
 - Throughput

- Power consumption (if available)
 - Compare with software baseline (CPU/GPU implementation)
- **Deliverables:**
 - Synthesis reports
 - Performance analysis document
 - Final presentation

Project Topics & Base Research Articles

1. KALMAN ALL THE WAY - Implementation of Multiple Types of Kalman Filters

Description: Implement various Kalman filter variants (Standard, Extended, Unscented) in hardware for real-time sensor fusion applications.

Key Research Articles:

1. Efficient Mapping of a Kalman Filter into an FPGA using Taylor Expansion

Liu, Yang, Christos-Savvas Bouganis, and Peter YK Cheung. "Efficient mapping of a Kalman filter into an FPGA using Taylor expansion." 2007 International Conference on Field Programmable Logic and Applications. IEEE, 2007.

2. FPGA implementation of multi-dimensional Kalman filter for object tracking and motion detection

Babu, Praveenkumar, and Eswaran Parthasarathy. "FPGA implementation of multi-dimensional Kalman filter for object tracking and motion detection." Engineering Science and Technology, an International Journal 33 (2022): 101084.

3. Parameterizable FPGA-based Kalman Filter Coprocessor Using Piecewise Affine Modeling

Mills, Aaron, Phillip H. Jones, and Joseph Zambreno. "Parameterizable FPGA-based Kalman filter coprocessor using piecewise affine modeling." 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE, 2016.

4. FPGA-based unscented Kalman filter for target tracking

AlShabi, Mohammad A., and Talal Bonny. "FPGA-based unscented Kalman filter for target

tracking." Signal Processing, Sensor/Information Fusion, and Target Recognition XXXI. Vol. 12122. SPIE, 2022.

5. **An FPGA Implementation for a Kalman Filter with Application to Mobile Robotics**

Bonato, Vanderlei, et al. "An fpga implementation for a kalman filter with application to mobile robotics." 2007 international symposium on industrial embedded systems. IEEE, 2007.

Implementation Suggestions:

- Start with basic Linear Kalman Filter
- Add Extended Kalman Filter (EKF) for nonlinear systems
- Implement Unscented Kalman Filter (UKF) if time permits
- Focus on fixed-point arithmetic optimization
- Use parallel processing for matrix operations

2. Optical Flow

Description: Implement real-time optical flow algorithms (Lucas-Kanade, Horn-Schunck) for motion estimation in video sequences.

Key Research Articles:

1. **Real-Time Efficient FPGA Implementation of the Multi-Scale Lucas-Kanade and Horn-Schunck Optical Flow Algorithms for a 4K Video Stream**

Blachut, Krzysztof, and Tomasz Kryjak. "Real-time efficient fpga implementation of the multi-scale lucas-kanade and horn-schunck optical flow algorithms for a 4k video stream." Sensors 22.13 (2022): 5017.

2. **Efficient Hardware Implementation of the Horn-Schunck Algorithm for Embedded Optical Flow Sensor**

Komorkiewicz, Mateusz, Tomasz Kryjak, and Marek Gorgon. "Efficient hardware implementation of the Horn-Schunck algorithm for high-resolution real-time dense optical flow sensor." Sensors 14.2 (2014): 2860-2891.

3. **Harms: A hardware acceleration architecture for real-time event-based optical flow**

Stumpp, Daniel C., et al. "Harms: A hardware acceleration architecture for real-time event optical flow." IEEE Access 10 (2022): 58181-58198.

4. **Flexible FPGA Acceleration Architecture for Real-Time Neuromorphic Optical Flow**

Silbernagel, Linus. Flexible FPGA Acceleration Architecture for Real-Time Neuromorphic Optical Flow. MS thesis. University of Pittsburgh, 2024.

5. **FPGA-based implementation of optical flow algorithm**

Allaoui, R., et al. "FPGA-based implementation of optical flow algorithm." 2017 international conference on electrical and information technologies (ICEIT). IEEE, 2017.

Implementation Suggestions:

- Begin with Horn-Schunck (easier to parallelize)
- Implement multi-scale pyramidal approach for large displacements
- Pipeline image derivatives computation
- Extra: Use fixed-point arithmetic (12-16 bits)

3. Clustering of Point Clouds

Description: Implement clustering algorithms (DBSCAN, Euclidean clustering) for 3D LiDAR point cloud segmentation.

Key Research Articles:

1. **Arc 2014: A multidimensional fpga-based parallel dbscan architecture**

Scicluna, Neil, and Christos-Savvas Bouganis. "Arc 2014: A multidimensional fpga-based parallel dbscan architecture." ACM Transactions on Reconfigurable Technology and Systems (TRETS) 9.1 (2015): 1-15.

2. **Real-Time LiDAR Point-Cloud Moving Object Segmentation for Autonomous Vehicles**

Xie, Xing, Haowen Wei, and Yongjie Yang. "Real-time LiDAR point-cloud moving object segmentation for autonomous driving." Sensors 23.1 (2023): 547.

3. **Stream-Based Ground Segmentation for Real-Time LiDAR Point Cloud Processing on FPGA**

Zhang, Xiao, et al. "Stream-Based Ground Segmentation for Real-Time LiDAR Point Cloud Processing on FPGA." arXiv preprint arXiv:2408.10410 (2024).

4. Real-Time Fast Channel Clustering for LiDAR Point Cloud

Zhang, Xiao, and Xinming Huang. "Real-time fast channel clustering for LiDAR point cloud." IEEE Transactions on Circuits and Systems II: Express Briefs 69.10 (2022): 4103-4107.

5. An Efficient FPGA Accelerator for Point Cloud Sparse Convolutional Networks

Wang, Zilun, et al. "An efficient fpga accelerator for point cloud." 2022 IEEE 35th International System-on-Chip Conference (SOCC). IEEE, 2022.

Implementation Suggestions:

- Start with Euclidean clustering (simpler than DBSCAN)
- Use K-d tree or octree spatial indexing
- Implement streaming architecture for real-time processing
- Support variable point cloud sizes
- Optimize memory access patterns

4. Motion Distortion Effect

Description: Implement LiDAR/camera motion distortion correction for moving platforms using IMU data.

Key Research Articles:

1. De-Skewing LiDAR Scan for Refinement of Local Mapping

He, Lei, Zhe Jin, and Zhenhai Gao. "De-skewing lidar scan for refinement of local mapping." Sensors 20.7 (2020): 1846.

2. Correcting Motion Distortion for LIDAR HD-Map Localization

McDermott, Matthew, and Jason Rife. "Correcting motion distortion for LIDAR scan-to-Map Registration." IEEE Robotics and Automation Letters 9.2 (2023): 1516-1523.

3. 3d lidar-imu calibration based on upsampled preintegrated measurements for motion distortion correction

Le Gentil, Cedric, Teresa Vidal-Calleja, and Shoudong Huang. "3d lidar-imu calibration based on upsampled preintegrated measurements for motion distortion correction." 2018 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2018.

4. **A robust adaptive unscented kalman filter for floating doppler wind-lidar motion correction**

Salcedo-Bosch, Andreu, Francesc Rocabdenbosch, and Joaquim Sospedra. "A robust adaptive unscented kalman filter for floating doppler wind-lidar motion correction." Remote Sensing 13.20 (2021): 4167.

5. **Increased Accuracy For Fast Moving LiDARS: Correction of Motion Distortion**

Renzler, Tobias, et al. "Increased accuracy for fast moving LiDARS: Correction of distorted point clouds." 2020 IEEE international instrumentation and measurement technology conference (I2MTC). IEEE, 2020.

Implementation Suggestions:

- Implement linear interpolation for motion compensation
- Use IMU integration for pose estimation
- Support rolling shutter correction
- Process point-by-point or scan-line-by-line
- Use quaternion-based rotation interpolation

5. Post-Quantum Algorithms (e.g., CRYSTALS)

Description: Implement post-quantum cryptographic algorithms (CRYSTALS-Kyber, CRYSTALS-Dilithium) resistant to quantum computer attacks.

Key Research Articles:

1. **High-speed hardware architectures and FPGA benchmarking of CRYSTALS-Kyber, NTRU, and Saber**

Dang, Viet Ba, Kamyar Mohajerani, and Kris Gaj. "High-speed hardware architectures and FPGA benchmarking of CRYSTALS-Kyber, NTRU, and Saber." IEEE transactions on computers 72.2 (2022): 306-320.

2. **Hardware Acceleration for High-Volume Operations of CRYSTALS-Kyber and CRYSTALS-Dilithium**

Carril, Xavier, et al. "Hardware acceleration for high-volume operations of CRYSTALS-kyber and CRYSTALS-dilithium." ACM Transactions on Reconfigurable Technology and Systems 17.3 : 1-26.

3. Efficient Hardware Implementations for Lattice-Based Cryptography Primitives

Mert, Ahmet Can. Efficient hardware implementations for lattice-based cryptography primitives. Diss. 2021.

4. KiD: A Hardware Design Framework Targeting Unified NTT for CRYSTALS-Kyber and Dilithium

Mandal, Suraj, and Debapriya Basu Roy. "Kid: A hardware design framework targeting unified ntt multiplication for crystals-kyber and crystals-dilithium on fpga." 2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID). IEEE, 2024.

5. Lightweight Hardware Implementation of R-LWE Lattice-Based Cryptography

Fan, Sailong, et al. "Lightweight hardware implementation of R-LWE lattice-based cryptography." 2018 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS). IEEE, 2018.

Implementation Suggestions:

- Focus on Number Theoretic Transform (NTT) acceleration
- Implement modular arithmetic units
- Use polynomial multiplication optimization
- Support multiple security levels (Kyber-512, Kyber-768, Kyber-1024)
- Optimize memory bandwidth for coefficient storage

6. Random Number Generation (NIST Compliant)

Description: Implement True Random Number Generator (TRNG) compliant with NIST SP 800-90A/B/C standards.

Key Research Articles:

1. FPGA-based True Random Number Generation using Circuit Metastability with Adaptive Feedback Control

Majzoobi, Mehrdad, Farinaz Koushanfar, and Srinivas Devadas. "FPGA-based true random number generation using circuit metastability with adaptive feedback control." International Workshop on Cryptographic Hardware and Embedded Systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011.

2. neoTRNG: A Tiny and Platform-Independent True Random Number Generator

Open Source Project

GitHub Repository

<https://github.com/stnolting/neoTRNG>

3. A metastability-based true random number generator on FPGA

Li, Chaoyang, et al. "A metastability-based true random number generator on FPGA." 2017 IEEE 12th international conference on ASIC (ASICON). IEEE, 2017.

4. NIST SP 800-90B Compliant Entropy Source Implementation

Xiphera Documentation

Commercial IP Documentation

<https://xiphera.com/random-number-generation/true-random-number-generation/>

5. True Random Number Generator Hardware (AN-1200)

Renesas Application Note

Technical Documentation

<https://www.renesas.com/us/en/document/apn/1200-true-random-number-generator-hardware>

Implementation Suggestions:

- Use ring oscillator or metastability-based entropy source
- Implement von Neumann corrector for bias removal
- Add NIST SP 800-22 statistical test interface
- Include health monitoring and startup tests

7. Quantization of AI Models on FPGA

Description: Implement neural network quantization techniques (INT8, INT4, binary) for efficient FPGA deployment.

Key Research Articles:

1. Trainable Fixed-Point Quantization for Deep Learning Acceleration on FPGAs

Dai, Dingyi, et al. "Trainable fixed-point quantization for deep learning acceleration on fp arXiv preprint arXiv:2401.17544 (2024).

2. **Post-training quantization for efficient FPGA-based neural network acceleration**

Salah, Oumayma Bel Haj, et al. "Post-training quantization for efficient FPGA-based neural network acceleration." Integration (2025): 102508.

3. **Quantized convolutional neural networks: a hardware perspective**

Zhang, Li, et al. "Quantized convolutional neural networks: a hardware perspective." Frontiers in Electronics 6 (2025): 1469802.

4. **Quantization-Aware NN Layers with High-throughput FPGA Implementation**

Pistellato, Mara, et al. "Quantization-aware nn layers with high-throughput fpga implementation for edge ai." Sensors 23.10 (2023): 4667.

5. **On-Chip Hardware-Aware Quantization for Mixed Precision Neural Networks**

Huang, Wei, et al. "On-Chip Hardware-Aware Quantization for Mixed Precision Neural Networks." arXiv preprint arXiv:2309.01945 (2023).

Implementation Suggestions:

- Start with 8-bit integer quantization
- Implement per-channel or per-layer quantization
- Support common layers (Conv2D, Dense, BatchNorm)
- Use fixed-point arithmetic throughout
- Compare accuracy vs. bit-width trade-offs

8. Large Number Representation System on FPGA/CUDA

Description: Implement arbitrary-precision arithmetic for cryptography and scientific computing.

Key Research Articles:

1. **ARCHITECT: Arbitrary-precision Hardware with Digit Elision for Efficient Iterative Compute**

Li, He, et al. "Architect: Arbitrary-precision hardware with digit elision for efficient iterative compute." IEEE Transactions on Very Large Scale Integration (VLSI) Systems 28.2 (2019): 516-529.

2. **FPGA implementation of the multiplication operation in multiple-precision arithmetic**

Rudnicki, Kamil, and Tomasz P. Stefański. "FPGA implementation of the multiplication operation in multiple-precision arithmetic." 2017 MIXDES-24th International Conference" Mixed Design of Integrated Circuits and Systems. IEEE, 2017.

3. **Fast Arbitrary Precision Floating Point on FPGA**

de Fine Licht, Johannes, et al. "Fast Arbitrary Precision Floating Point on FPGA." 2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). IEEE, 2022.

4. **GPU Implementations for Midsize Integer Addition and Multiplication**

Oancea, Cosmin E., and Stephen M. Watt. "GPU Implementations for Midsize Integer Addition and Multiplication." arXiv preprint arXiv:2405.14642 (2024).

5. **Big Integer Multiplication with CUDA FFT (cuFFT) Library**

Bantikyan, Hovhannes. "Big integer multiplication with CUDA FFT (cuFFT) library." (2014).

Implementation Suggestions:

- Implement basic operations (add, subtract, multiply)
- Use Karatsuba or FFT-based multiplication for large numbers
- Support variable precision (64-bit to 8192-bit)
- Optimize DSP block utilization
- Consider Montgomery multiplication for modular arithmetic

9. Point Cloud-RGB Calibration

Description: Implement extrinsic calibration between LiDAR sensors and RGB cameras for sensor fusion.

Key Research Articles:

1. **FAST-Calib: LiDAR-Camera Extrinsic Calibration in One Second**

Zheng, Chunran, and Fu Zhang. "FAST-Calib: LiDAR-Camera Extrinsic Calibration in One Second." arXiv preprint arXiv:2507.17210 (2025).

2. **Extrinsic Calibration of 3D Sensors Using a Spherical Target**

Ruan, Minghao, and Daniel Huber. "Calibration of 3D sensors using a spherical target." 2014 2nd International Conference on 3D Vision. Vol. 1. IEEE, 2014.

3. **What Is Lidar-Camera Calibration?**

MATLAB Documentation (2024)

MathWorks Technical Documentation

<https://www.mathworks.com/help/lidar/ug/lidar-camera-calibration.html>

4. **External multi-modal imaging sensor calibration for sensor fusion**

Qiu, Zhouyan, et al. "External multi-modal imaging sensor calibration for sensor fusion: A review." *Information Fusion* 97 (2023): 101806.

5. **2D LiDAR and Camera Fusion in 3D Modeling of Indoor Environments**

Li, Juan, Xiang He, and Jia Li. "2D LiDAR and camera fusion in 3D modeling of indoor environment." 2015 National Aerospace and Electronics Conference (NAECON). IEEE, 2015.

6. **LiDAR-camera calibration using 3D-3D point correspondences**

Dhall, Ankit, et al. "LiDAR-camera calibration using 3D-3D point correspondences." *arXiv preprint arXiv:1705.09785* (2017).

Implementation Suggestions:

- Implement checkerboard or circle grid detection
- Use PnP (Perspective-n-Point) algorithm
- Calculate rotation and translation matrices
- Support multiple calibration patterns
- Implement outlier rejection (RANSAC)

10. SLAM (Simultaneous Localization and Mapping)

Description: Implement real-time SLAM algorithms for autonomous navigation using LiDAR or cameras.

Key Research Articles:

1. **Energy-efficient FPGA-accelerated LiDAR-based SLAM for Embedded Robotics**

Flottmann, Marcel, et al. "Energy-efficient FPGA-accelerated LiDAR-based SLAM for embedded robotics." 2021 International Conference on Field-Programmable Technology (ICFPT). IEEE, 2021.

2. **A Universal LiDAR SLAM Accelerator System on Low-cost FPGA**

Sugiura, Keisuke, and Hiroki Matsutani. "A universal LiDAR SLAM accelerator system on low-cost FPGA." IEEE Access 10 (2022): 26931-26947.

3. **Accelerated Feature Detectors for Visual SLAM: A Comparative Study of FPGA vs GPU**

Ye, Ruiqi, and Mikel Luján. "Accelerated Feature Detectors for Visual SLAM: A Comparative Study of FPGA vs GPU." arXiv preprint arXiv:2510.13546 (2025).

4. **An Edge-Server Partitioning Method for 3D LiDAR SLAM on FPGAs**

Yasuda, Mizuki, et al. "An Edge-Server Partitioning Method for 3D LiDAR SLAM on FPGAs." 2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE, 2023.

5. **An FPGA-based Real-time Simultaneous Localization and Mapping System**

Gu, Mengyuan, et al. "An FPGA-based real-time simultaneous localization and mapping system." 2015 International Conference on Field Programmable Technology (FPT). IEEE, 2015.

Implementation Suggestions:

- Start with 2D LiDAR SLAM (simpler than 3D)
- Implement scan matching (ICP or correlative)
- Use occupancy grid or TSDF map representation
- Add loop closure detection for drift correction

11. Ground Plane Detection and Segmentation

Description: Implement real-time ground plane segmentation for LiDAR point clouds using channel-based methods or RANSAC for autonomous navigation and obstacle detection.

Key Research Articles:

1. **Stream-Based Ground Segmentation for Real-Time LiDAR Point Cloud Processing on FPGA**

Zhang, Xiao, et al. "Stream-Based Ground Segmentation for Real-Time LiDAR Point Cloud Processing on FPGA." arXiv preprint arXiv:2408.10410 (2024).

2. Ground Plane Segmentation of Lidar Data on FPGA

MathWorks (2024)

Technical Documentation

<https://www.mathworks.com/help/visionhdl/ug/lidar-ground-segmentation.html>

Savitzky-Golay smoothing with breadth-first search labeling

3. RANSAC for Robotic Applications: A Survey

Martínez-Otzeta, José María, et al. "Ransac for robotic applications: A survey." Sensors 23.1 (2022): 327.

4. A Survey of RANSAC Enhancements for Plane Detection in 3D Point Clouds

Zeineldin, Ramy Ashraf, and Nawal Ahmed El-Fishawy. "A survey of RANSAC enhancements for plane detection in 3D point clouds." Menoufia J. Electron. Eng. Res 26.2 (2017): 519-537.

5. Adaptive Ground Segmentation Method for Real-time Mobile Robot Applications

Vu, Hoang, et al. "Adaptive ground segmentation method for real-time mobile robot control." International Journal of Advanced Robotic Systems 14.6 (2017): 1729881417748135.

Implementation Suggestions:

- Start with channel-based approach (easier than RANSAC)
- Implement Savitzky-Golay filter for range smoothing
- Use flood-fill algorithm for ground labeling

12. 3D Object Detection from LiDAR Point Clouds

Description: Implement voxel-based or pillar-based 3D object detection.

Key Research Articles:

1. LiFT: Lightweight, FPGA-tailored 3D Object Detection Based on LiDAR Data

Lis, Konrad, Tomasz Kryjak, and Marek Gorgoń. "LiFT: Lightweight, FPGA-tailored 3D object detection based on LiDAR data." International Workshop on Design and Architectures for Signal and Image Processing. Cham: Springer Nature Switzerland, 2025.

2. LiDAR 3D Object Detection in FPGA with Low Bitwidth Quantization

Brum, Henrique, Mário Véstias, and Horácio Neto. "LiDAR 3D object detection in FPGA with low bitwidth quantization." International Symposium on Applied Reconfigurable Computing. Cham: Springer Nature Switzerland, 2024.

3. VEA: An FPGA-Based Voxel Encoding Accelerator for 3D Object Detection

Li, Xin, et al. "Vea: An fpga-based voxel encoding accelerator for 3d object detection with lidar." 2022 IEEE 40th International Conference on Computer Design (ICCD). IEEE, 2022.

4. IBB-Net: Fast Iterative Bounding Box Regression for Detection on Point Clouds

Miller, Brendan. IBB-Net: Fast Iterative Bounding Box Regression for Detection on Point Clouds. Diss. Master's thesis, Pittsburgh, PA (June 2020), 2020.

5. Deep Sensor Fusion for 3D Bounding Box Estimation

Xu, Danfei, Dragomir Anguelov, and Ashesh Jain. "Pointfusion: Deep sensor fusion for 3d bounding box estimation." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

Implementation Suggestions:

- Focus on pillar-based approach (simpler than full 3D voxels)
- Use 2D convolutions on bird's-eye-view representation
- Implement non-maximum suppression (NMS) in hardware

General Guidelines for All Projects

Documentation Phase Tips

- Use IEEE Xplore, ACM Digital Library, arXiv for papers
- Focus on recent papers (2015-2025) for state-of-the-art
- Include both algorithmic and implementation papers
- Explain mathematical foundations clearly
- Discuss hardware-software trade-offs

Implementation Phase Tips

- Start with functional simulation before synthesis
- Use modular design for easier debugging
- Document all interfaces and protocols
- Include comprehensive testbenches
- Use version control (Git) from day one
- Write clear README with build instructions

Evaluation Phase Tips

- Use realistic test vectors/datasets
- Compare against reference implementations
- Report both resource utilization and performance
- Analyze power consumption if tools support it
- Create graphs and tables for results
- Discuss limitations and future improvements

FPGA Tools and Platforms

- **Xilinx:** Vivado (for Artix-7)
- **Open-Source:** Icarus Verilog, Verilator
- **Synthesis:** Vivado Synthesis, Yosys
- **Place & Route:** Vivado Implementation, nextpnr
- **Simulation:** ModelSim, Vivado Simulator, Gtkwave
- **HLS:** Vitis HLS, HLSLibs, ScaleHLS

Recommended Development Flow

1. Algorithm understanding and MATLAB/Python prototype
2. HDL implementation with testbench
3. Functional simulation and verification
4. Synthesis and timing analysis
5. FPGA implementation and hardware testing
6. Performance evaluation and optimization

Grading Criteria (Approximate)

- **Documentation:** 10
 - Literature review quality
 - Problem understanding
 - Presentation clarity
- **Implementation:** 10
 - Code quality and organization
 - Functionality and correctness
 - Testing and verification
- **Evaluation:** 10
 - Synthesis results
 - Performance analysis
 - Comparison with baselines

Important Deadlines

- **Documentation Phase:** 26.11.2025, 23:59
- **Implementation Phase:** 17.12.2025, 23:59
- **Final Evaluation:** 14.01.2026, 23:59

Additional Resources

Online Courses and Tutorials

- **FPGAs for Beginners** (Nandland, YouTube)
- **Hardware Description Languages** (HDLBits)
- **Chisel Bootcamp** (UC Berkeley)
- **FPGA Design Patterns**

Useful Websites

- **IEEE Xplore:** <https://ieeexplore.ieee.org>
- **ACM Digital Library:** <https://dl.acm.org>
- **arXiv.org:** <https://arxiv.org>
- **FPGA4Student:** <https://www.fpga4student.com>
- **GitHub:** Search for "FPGA + [your topic]"

Books

- Digital Design and Computer Architecture
- FPGAs: Fundamentals, Advanced Features, and Applications in Industrial Electronics
- RTL Modeling with SystemVerilog for ASIC and FPGA Design
- High-Level Synthesis: from Algorithm to Digital Circuit